COMPETITIVE PROGRAMMING LIBRARY

YOUR NAME | YOUR TEAM

Compiled on: 2026-03-14

INDEX

DYNAMIC-PROGRAMMING

divide-and-conquer.cpp 02
knuth.cpp 02
sum-over-subsets.cpp 02
sum-over-supersets.cpp 02

EXTRA

nim.cpp 02
offset.cpp 02
out.cpp 02
pragma.cpp 02
random.cpp 02

GEOMETRY

polygons.cpp 02
primitives.cpp 02

GRAPH

articulations.cpp 03
bellman-ford.cpp 03
bridges.cpp 03
cost-flow.cpp 03
cycle.cpp 03
dijkstra.cpp 04
dinic.cpp 04
floyd-warshall.cpp 04
johnson.cpp 04
tarjan.cpp 04
two-sat.cpp 04

GRAPH/TREE

binary-lifting.cpp 04
centroid-decomposition.cpp 05
heavy-light-decomposition.cpp 05

MATH

matrix.cpp 05
xor-basis.cpp 05

MATH/CONVOLUTION

fast-fourier-transform.cpp 06
fast-walsh-hadamard-transform.cpp 06
fast-zeta-transform.cpp 06
number-theoretic-transform.cpp 06

NUMBER-THEORY

mobius.cpp 06
totient.cpp 07

NUMBER-THEORY/MOD

chinese-remainder-theorem.cpp 07
extended-gcd.cpp 07
log.cpp 07
multi-extended-gcd.cpp 07
power.cpp 07

NUMBER-THEORY/PRIME

linear-sieve.cpp 07
miller-rabin.cpp 07
segmented-sieve.cpp 07

SEARCH

ternary-search.cpp 07

SQUARE-ROOT

decomposition.cpp 08
mo.cpp 08

STRING

aho-corasick.cpp 08
hasher-segment-tree.cpp 08
hasher.cpp 08
longest-prefix-suffix.cpp 09
manacher.cpp 09
palindromic-tree.cpp 09
suffix-automaton.cpp 09
trie.cpp 09
z-array.cpp 10

STRUCTURE

coprime-set.cpp 10
cover-tree.cpp 10
disjoint-set-union.cpp 10
line-container.cpp 10
merge-sort-tree.cpp 10
range-set.cpp 10
sparse-table.cpp 11
treap.cpp 11
venice-set.cpp 11
wavelet-tree.cpp 12

STRUCTURE/BINARY-INDEXED-TREE

binary-indexed-tree-2d.cpp 12
binary-indexed-tree.cpp 12
sparse-binary-indexed-tree.cpp 12

STRUCTURE/POLICY-BASED-DATA-STRUCTURES

hash-table.cpp 12
ordered-set.cpp 12

STRUCTURE/SEGMENT-TREE

lazy-segment-tree.cpp 12
persistent-segment-tree.cpp 13
segment-tree.cpp 13
sparse-segment-tree.cpp 13

DYNAMIC-PROGRAMMING

divide-and-conquer.cpp

```cpp
int divide_and_conquer() {
    vector<int> prev(n + 1, inf);
    vector<int> curr(n + 1, inf);
    function<void(int, int, int, int)> compute = [&](int l, int r, int opt_l, int opt_r) ->
void {
        if (l <= r) {
            int m = (l + r) / 2;
            curr[m] = inf;
            int opt = opt_l;
            for (int k = opt_l; k <= min(m - 1, opt_r); ++k) {
                int val = prev[k] + cost(k + 1, m);
                if (val < curr[m]) {
                    curr[m] = val;
                    opt = k;
                }
            }
            compute(l, m - 1, opt_l, opt);
            compute(m + 1, r, opt, opt_r);
        }
    };
    prev[0] = 0;
    for (int i = 1; i <= k; ++i) {
        compute(1, n, 0, n - 1);
        prev = curr;
    }
    return curr[n];
}
```

knuth.cpp

```cpp
int knuth() {
    vector<vector<int>> val(n + 5, vector<int>(n + 5));
    vector<vector<int>> opt(n + 5, vector<int>(n + 5));
    for (int i = 1; i <= n; ++i) {
        val[i][i] = cost(i, i);
        opt[i][i] = i;
    }
    for (int len = 2; len <= n; ++len) {
        for (int l = 1, r = len; r <= n; ++l, ++r) {
            val[l][r] = inf;
            opt[l][r] = opt[l][r - 1];
            for (int k = opt[l][r - 1]; k <= opt[l + 1][r]; ++k) {
                int curr = val[l][k] + val[k + 1][r] + cost(l, r);
                if (curr < val[l][r]) {
                    val[l][r] = curr;
                    opt[l][r] = k;
                }
            }
        }
    }
    return val[1][n];
}
```

sum-over-subsets.cpp

```cpp
vector<int> sum_over_subsets(int n, vector<int> f) {
    int full = (1 << n) - 1;
    for (int i = 0; i < n; ++i) {
        int bit = 1 << i;
        for (int mask = 0; mask <= full; ++mask) if (mask & bit) {
            f[mask] += f[mask ^ bit];
        }
    }
    return f;
}
```

sum-over-supersets.cpp

```cpp
vector<int> sum_over_supersets(int n, vector<int> f) {
    int full = (1 << n) - 1;
    for (int i = 0; i < n; ++i) {
        int bit = 1 << i;
        for (int mask = 0; mask <= full; ++mask) if (not (mask & bit)) {
            f[mask] += f[mask | bit];
        }
    }
    return f;
}
```

EXTRA

nim.cpp

```cpp
bool nim(vector<int> &piles, vector<int> &moves) {
    int n = *max(piles.begin(), piles.end());
    vector<int> grundy(n + 1);

    for (int i = 1; i <= n; ++i) {
        set<int> reach;
        for (int move : moves) if (move <= i) {
            reach.insert(grundy[i - move]);
        }

        int mex = 0;
        while (reach.contains(mex)) mex++;
        grundy[i] = mex;
    }

    int answer = 0;
    for (int pile : piles) {
        answer ^= grundy[pile];
    }
    return answer != 0;
}

bool misere(vector<int> &piles, vector<int> &moves) {
    int n = *max(piles.begin(), piles.end());
    vector<int> grundy(n + 1);

    for (int i = 1; i <= n; ++i) {
        set<int> reach;
        for (int move : moves) if (move <= i) {
            reach.insert(grundy[i - move]);
        }

        int mex = 0;
        while (reach.contains(mex)) mex++;
        grundy[i] = mex;
    }

    bool all_one = true;
    int answer = 0;
    for (int pile : piles) {
        all_one &= grundy[pile] == 1;
        answer ^= grundy[pile];
    }
    return all_one ? (piles.size() & 1) == 0 : answer != 0;
}
```

offset.cpp

```cpp
template <typename T>
struct offset {
    int l;
    vector<T> a;

    offset(int l, int r, T x = T()): l(l), a(r - l, x) {}

    T &operator[](int i) {
        return a[i - l];
    }

    const T &operator[](int i) const {
        return a[i - l];
    }
};
```

out.cpp

```cpp
template <typename T>
ostream &operator<<(ostream &out, const vector<T> &a) {
    for (int i = 0; i < a.size(); ++i) {
        if (i != 0) out << ' ';
        out << a[i];
    }
    return out;
}
```

pragma.cpp

```cpp
#pragma GCC optimize("Ofast")
```

random.cpp

```cpp
mt19937 random(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution<int> distribution(l, r);
```

GEOMETRY

polygons.cpp

```cpp
float area(vector<point> &p) {
    int n = p.size();
    float a = 0;
    for (int i = 0; i < n; ++i) {
        int j = (i + 1) % n;
        a += p[i].x * p[j].y - p[j].x * p[i].y;
    }
    return a / 2;
}

vector<point> convex_hull(vector<point> &p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());

    int n = p.size();
    if (n < 3) return p;

    bool collinear = true;
    line h = {p[0], p[1]};
    for (int i = 2; i < n and collinear; ++i) collinear &= h.orient(p[i]) == 0;
    if (collinear) return {p[0], p[n - 1]};

    vector<point> lower;
    for (int i = 0; i < n; ++i) {
        while (lower.size() >= 2) {
            line l = {lower[lower.size() - 2], lower[lower.size() - 1]};
            if (l.orient(p[i]) == -1) break;
            lower.pop_back();
        }
        lower.push_back(p[i]);
    }

    vector<point> upper;
    for (int i = n - 1; i >= 0; --i) {
        while (upper.size() >= 2) {
            line l = {upper[upper.size() - 2], upper[upper.size() - 1]};
            if (l.orient(p[i]) == -1) break;
            upper.pop_back();
        }
        upper.push_back(p[i]);
    }

    lower.pop_back();
    upper.pop_back();

    vector<point> hull = lower;
    hull.insert(hull.end(), upper.begin(), upper.end());
    return hull;
}
```

primitives.cpp

```cpp
struct point {
    float x, y;

    bool operator==(const point &other) const {
        return x == other.x and y == other.y;
    }

    bool operator<(const point &other) const {
        return tie(x, y) < tie(other.x, other.y);
    }

    point operator-(const point &other) const {
        return {x - other.x, y - other.y};
    }
```

```
    static float dist(point &a, point &b) {
        float dx = a.x - b.x;
        float dy = a.y - b.y;
        return sqrt(dx * dx + dy * dy);
    }

    static float dot(point &a, point &b) {
        return a.x * b.x + a.y * b.y;
    }
};

struct line {
    point a, b;

    int orientation(point &p) {
        point v = b - a;
        point w = p - a;

        float c = v.x * w.y - v.y * w.x;
        if (c > 0) return -1;
        if (c < 0) return +1;
        return 0;
    }

    bool between(point &p) {
        if (orientation(p) != 0) return false;

        bool bx = min(a.x, b.x) <= p.x and p.x <= max(a.x, b.x);
        bool by = min(a.y, b.y) <= p.y and p.y <= max(a.y, b.y);
        return bx and by;
    }

    point project(point &p) {
        point v = b - a;
        point w = p - a;

        float t = point::dot(v, w) / point::dot(v, v);
        if (t < 0) t = 0;
        if (t > 1) t = 1;

        return {a.x + t * v.x, a.y + t * v.y};
    }

    float dist(point &p) {
        point q = project(p);
        return point::dist(p, q);
    }

    line normal() {
        point v = b - a;
        v = {-v.y, v.x};

        point c = {(a.x + b.x) / 2, (a.y + b.y) / 2};
        point d = {c.x + v.x, c.y + v.y};
        return {c, d};
    }

    static float angle(line &s, line &t) {
        point v = s.b - s.a;
        point w = t.b - t.a;

        float c = point::dot(v, w) / sqrt(point::dot(v, v) * point::dot(w, w));
        if (c < -1) c = -1;
        if (c > 1) c = 1;

        return acos(c);
    }

    static bool intersect(line &s, line &t) {
        if (s.between(t.a) or s.between(t.b)) return true;
        if (t.between(s.a) or t.between(s.b)) return true;

        if (s.orientation(t.a) == s.orientation(t.b)) return false;
        if (t.orientation(s.a) == t.orientation(s.b)) return false;

        return true;
    }

    static float dist(line &s, line &t) {
        if (line::intersect(s, t)) return 0;
        return min({s.dist(t.a), s.dist(t.b), t.dist(s.a), t.dist(s.b)});
    }
};
```

GRAPH

articulations.cpp

```
vector<int> articulations(int one, vector<vector<int>> &g) {
    int n = g.size(), timer = 1;
    vector<int> in(n), low(n);
    vector<bool> art(n);

    auto dfs = [&](auto &self, int a, int p) -> void {
        in[a] = low[a] = timer++;
        int c = 0;
        for (int b : g[a]) if (b != p) {
            if (in[b] != 0) {
                low[a] = min(low[a], in[b]);
            } else {
                c++;
                self(self, b, a);
                low[a] = min(low[a], low[b]);
                if (a != p and low[b] >= in[a]) art[a] = true;
            }
        }
        if (a == p and c > 1) art[a] = true;
    };

    vector<int> points;
    for (int i = one; i < n; ++i) {
        if (in[i] == 0) dfs(dfs, i, i);
        if (art[i]) points.push_back(i);
    }
    return points;
}
```

bellman-ford.cpp

```
vector<int> bellman_ford(int s, int n, vector<array<int, 3>> &e) {
    vector<int> d(n, inf);
    d[s] = 0;
    for (int i = 1; i < n; ++i) {
        bool chd = false;
        for (auto [a, b, w] : e) if (d[a] != inf and d[a] + w < d[b]) {
            d[b] = d[a] + w;
            chd = true;
        }
        if (not chd) break;
    }
    for (auto [a, b, w] : e) if (d[a] != inf and d[a] + w < d[b]) return {};
    return d;
}

vector<int> shortest_path_faster_algorithm(int s, vector<vector<pair<int, int>>> &g) {
    int n = g.size();
    vector<int> d(n, inf), c(n);
    vector<bool> in(n);
    queue<int> q;
    d[s] = 0;
    in[s] = true;
    q.push(s);
    while (q.size()) {
        int a = q.front(); q.pop();
        in[a] = false;
        for (auto [b, w] : g[a]) if (d[a] + w < d[b]) {
            d[b] = d[a] + w;
            if (not in[b]) {
                c[b] += 1;
                if (c[b] >= n) return {};
                in[b] = true;
                q.push(b);
            }
        }
    }
    return d;
}
```

bridges.cpp

```
vector<pair<int, int>> bridges(int one, vector<vector<int>> &g) {
    int n = g.size(), timer = 1;
    vector<int> in(n), low(n);
    vector<pair<int, int>> e;

    auto dfs = [&](auto &self, int a, int p) -> void {
        in[a] = low[a] = timer++;
        for (int b : g[a]) if (b != p) {
            if (in[b] != 0) {
                low[a] = min(low[a], in[b]);
            } else {
                self(self, b, a);
                low[a] = min(low[a], low[b]);
                if (in[a] < low[b]) e.emplace_back(a, b);
            }
        }
    };

    for (int i = one; i < n; ++i) if (in[i] == 0) dfs(dfs, i, i);
    return e;
}
```

cost-flow.cpp

```
struct min_cost_flow {
    int n;
    vector<vector<array<int, 4>>> g;
    vector<int> d, h, p, e;

    min_cost_flow(int n): n(n), g(n), d(n), h(n), p(n), e(n) {}

    void insert_edge(int a, int b, int capacity, int cost) {
        g[a].push_back({b, g[b].size(), capacity, cost});
        g[b].push_back({a, g[a].size() - 1, 0, -cost});
    }

    bool dijkstra(int s, int t) {
        fill(d.begin(), d.end(), inf);
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
        d[s] = 0;
        pq.emplace(0, s);
        while (pq.size()) {
            auto [x, a] = pq.top(); pq.pop();
            if (d[a] < x) continue;

            for (int i = 0; i < g[a].size(); ++i) {
                auto &[b, j, capacity, cost] = g[a][i];
                if (capacity <= 0) continue;

                int w = d[a] + cost + h[a] - h[b];
                if (w < d[b]) {
                    d[b] = w;
                    p[b] = a;
                    e[b] = i;
                    pq.emplace(d[b], b);
                }
            }
        }
        return d[t] != inf;
    }

    pair<int, int> min_cost_k_flow(int s, int t, int k = inf) {
        int flow = 0, cost = 0;
        fill(h.begin(), h.end(), 0);
        while (flow < k and dijkstra(s, t)) {
            for (int a = 0; a < n; ++a) if (d[a] != inf) {
                h[a] += d[a];
            }

            int push = k - flow;
            for (int a = t; a != s; a = p[a]) {
                int b = p[a], i = e[a];
                push = min(push, g[b][i][2]);
            }
            for (int a = t; a != s; a = p[a]) {
                int b = p[a], i = e[a], j = g[b][i][1];
                g[b][i][2] -= push;
                g[a][j][2] += push;
                cost += g[b][i][3] * push;
            }
            flow += push;
        }
        return {flow, cost};
    }
};
```

cycle.cpp

```
vector<int> find_cycle(bool one, bool is_directed, vector<vector<int>> &g) {
    int n = g.size();
    vector<int> state(n), parent(n);
    vector<int> cycle;
    auto dfs = [&](auto &self, int a, int p) -> bool {
        state[a] = 1;
        for (int b : g[a]) if (b != p or is_directed) {
            if (state[b] == 0) {
                parent[b] = a;
                if (self(self, b, a)) return true;
            } else if (state[b] == 1) {
```

```
                int c = a;
                while (c != b) {
                    cycle.push_back(c);
                    c = parent[c];
                }
                cycle.push_back(b);
                reverse(cycle.begin(), cycle.end());
                return true;
            }
        }
        state[a] = 2;
        return false;
    };
    for (int i = one; i < n; ++i) if (state[i] == 0) {
        if (dfs(dfs, i, -1)) return cycle;
    }
    return {};
}
```

dijkstra.cpp

```
vector<int> dijkstra(int s, vector<vector<pair<int, int>>> &g) {
    int n = g.size();
    vector<int> d(n, inf);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
    d[s] = 0;
    pq.emplace(0, s);
    while (pq.size()) {
        auto [x, a] = pq.top(); pq.pop();
        if (d[a] < x) continue;
        for (auto [b, w] : g[a]) if (d[a] + w < d[b]) {
            d[b] = d[a] + w;
            pq.emplace(d[b], b);
        }
    }
    return d;
}
```

dinic.cpp

```
struct dinic {
    int n;
    vector<vector<array<int, 3>>> g;
    vector<int> l, p;

    dinic(int n): n(n), g(n), l(n), p(n) {}

    void insert(int a, int b, int w) {
        g[a].push_back({b, w, g[b].size()});
        g[b].push_back({a, 0, g[a].size() - 1});
    }

    bool bfs(int s, int t) {
        fill(l.begin(), l.end(), -1);
        l[s] = 0;
        queue<int> q;
        q.push(s);

        while (not q.empty()) {
            int a = q.front(); q.pop();
            for (auto &[b, w, i] : g[a]) {
                if (w > 0 and l[b] < 0) {
                    l[b] = l[a] + 1;
                    q.push(b);
                }
            }
        }
        return l[t] >= 0;
    }

    int dfs(int a, int t, int f) {
        if (a == t) return f;

        for (int &k = p[a]; k < g[a].size(); ++k) {
            auto &[b, w, i] = g[a][k];
            if (w > 0 and l[b] == l[a] + 1) {
                int d = 0;
                if ((d = dfs(b, t, min(f, w))) > 0) {
                    w -= d;
                    g[b][i][1] += d;
                    return d;
                }
            }
        }
        return 0;
    }

    int flow(int s, int t) {
        int f = 0;
        while (bfs(s, t)) {
            fill(p.begin(), p.end(), 0);
            int d = 0;
            while ((d = dfs(s, t, inf)) > 0) {
                f += d;
            }
        }
        return f;
    }
};
```

floyd-warshall.cpp

```
vector<vector<int>> floyd_warshall(int n, vector<vector<int>> &g) {
    vector<vector<int>> d = g;
    for (int i = 0; i < n; ++i) {
        d[i][i] = 0;
    }
    for (int k = 0; k < n; ++k) {
        for (int i = 0; i < n; ++i) if (d[i][k] != inf) {
            for (int j = 0; j < n; ++j) if (d[k][j] != inf) {
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
            }
        }
    }
    return d;
}
```

johnson.cpp

```
vector<vector<int>> johnson(vector<vector<pair<int, int>>> &g) {
    int n = g.size();
    g.emplace_back();
    for (int i = 0; i < n; ++i) {
        g[n].emplace_back(i, 0);
    }
    vector<int> h = shortest_path_faster_algorithm(n, g);
    g.pop_back();
    if (h.empty()) return {};
    h.pop_back();
    for (int i = 0; i < n; ++i) {
        for (auto &[j, w] : g[i]) {
            w += h[i] - h[j];
        }
    }
    vector<vector<int>> d;
    for (int i = 0; i < n; ++i) {
        d.push_back(dijkstra(i, g));
        for (int j = 0; j < n; ++j) {
            if (d[i][j] != inf) d[i][j] -= h[i] - h[j];
        }
    }
    return d;
}
```

tarjan.cpp

```
vector<vector<int>> tarjan(int r, vector<vector<int>> &g) {
    int n = g.size(), timer = 1;
    vector<vector<int>> components;
    vector<int> id(n), low(n);
    stack<int> trail;
    vector<bool> in(n);

    auto dfs = [&](auto &self, int a) -> void {
        id[a] = low[a] = timer++;
        trail.push(a);
        in[a] = true;
        for (int b : g[a]) {
            if (id[b] == 0) {
                self(self, b);
                low[a] = min(low[a], low[b]);
            } else if (in[b]) {
                low[a] = min(low[a], id[b]);
            }
        }

        if (id[a] == low[a]) {
            vector<int> component;
            while (true) {
                int b = trail.top(); trail.pop();
                in[b] = false;
                component.push_back(b);
                if (b == a) break;
            }
            components.push_back(component);
        }
    };

    for (int a = r; a < n; ++a) {
        if (id[a] == 0) {
            dfs(dfs, a);
        }
    }

    reverse(components.begin(), components.end());
    return components;
}

pair<vector<vector<int>>, vector<int>> condensed_graph(int r, vector<vector<int>> &g) {
    int n = g.size();
    vector<vector<int>> components = tarjan(r, g);
    int m = components.size();

    vector<int> id(n);
    for (int i = 0; i < m; ++i) {
        for (int a : components[i]) {
            id[a] = i;
        }
    }

    vector<vector<int>> c(m);
    for (int a = r; a < n; ++a) {
        for (int b : g[a]) if (id[a] != id[b]) {
            c[id[a]].push_back(id[b]);
        }
    }

    for (int i = 0; i < m; ++i) {
        sort(c[i].begin(), c[i].end());
        c[i].erase(unique(c[i].begin(), c[i].end()), c[i].end());
    }
    return {c, id};
}
```

two-sat.cpp

```
struct two_sat {
    int n;
    vector<vector<int>> g;

    two_sat(int n): n(n), g(2 * n) {}

    void disjunction(int a, bool n_a, int b, bool n_b) {
        a = (2 * a) ^ n_a;
        b = (2 * b) ^ n_b;
        g[a ^ 1].push_back(b);
        g[b ^ 1].push_back(a);
    }

    vector<bool> build() {
        vector<vector<int>> components = tarjan(g, 0);
        int m = components.size();
        vector<int> id(2 * n);
        for (int i = 0; i < m; ++i) {
            for (int a : components[i]) {
                id[a] = i;
            }
        }
        vector<bool> assignment(n);
        for (int i = 0; i < n; ++i) {
            if (id[2 * i] == id[(2 * i) ^ 1]) return {};
            assignment[i] = id[2 * i] > id[(2 * i) ^ 1];
        }
        return assignment;
    }
};
```

GRAPH/TREE

binary-lifting.cpp

```
struct binary_lifting {
    int n, log;
    vector<int> d;
```

```
    vector<vector<int>> up;

    binary_lifting(int r, vector<vector<int>> &g): n(g.size()), log(31 - __builtin_clz(n)),
d(n), up(n, vector<int>(log + 1)) {
        d[r] = 0;
        build(r, 0, g);
    }

    void build(int a, int p, vector<vector<int>> &g) {
        up[a][0] = p;
        for (int i = 1; i <= log; ++i) {
            up[a][i] = up[up[a][i - 1]][i - 1];
        }

        for (int b : g[a]) if (b != p) {
            d[b] = d[a] + 1;
            build(b, a, g);
        }
    }

    int kth_ancestor(int a, int k) {
        if (d[a] < k) return -1;

        for (int i = log; i >= 0; --i) {
            if (k & (1 << i)) {
                a = up[a][i];
            }
        }
        return a;
    }

    int lowest_common_ancestor(int a, int b) {
        if (d[a] < d[b]) swap(a, b);
        a = kth_ancestor(a, d[a] - d[b]);
        if (a == b) return a;

        for (int i = log; i >= 0; --i) {
            if (up[a][i] != up[b][i]) {
                a = up[a][i];
                b = up[b][i];
            }
        }
        return up[a][0];
    }

    int kth_node(int a, int b, int k) {
        int lca = lowest_common_ancestor(a, b);
        int l = d[a] - d[lca];
        int r = d[b] - d[lca];
        if (k > l + r) return -1;

        if (k <= l) return kth_ancestor(a, k);
        return kth_ancestor(b, r - (k - l));
    }
};
```

centroid-decomposition.cpp

```
pair<vector<int>, vector<vector<int>>> centroid_decomposition(int r, vector<vector<int>> &g
) {
    int n = g.size();
    vector<int> parent(n);
    vector<int> size(n);
    vector<bool> removed(n);
    vector<vector<int>> dist(n);
    auto dfs = [&](auto &self, int a, int p) -> int {
        size[a] = 1;
        for (int b : g[a]) if (b != p and not removed[b]) {
            size[a] += self(self, b, a);
        }
        return size[a];
    };
    auto centroid = [&](auto &self, int a, int p, int s) -> int {
        for (int b : g[a]) if ((b != p and not removed[b]) and size[b] > s / 2) {
            return self(self, b, a, s);
        }
        return a;
    };
    auto distance = [&](auto &self, int a, int p, int d) -> void {
        dist[a].push_back(d);
        for (int b : g[a]) if (b != p and not removed[b]) {
            self(self, b, a, d + 1);
        }
    };
    auto build = [&](auto &self, int a, int p) -> void {
        int s = dfs(dfs, a, a);
        a = centroid(centroid, a, a, s);
        distance(distance, a, a, 0);
        parent[a] = p;
        removed[a] = true;
        for (int b : g[a]) if (not removed[b]) {
            self(self, b, a);
        }
    };
    build(build, r, -1);
    for (auto &path : dist) reverse(path.begin(), path.end());
    return {parent, dist};
}
```

heavy-light-decomposition.cpp

```
struct heavy_light_decomposition {
    int n, timer = 0;
    vector<vector<int>> &g;
    vector<int> p, d, h;
    segment_tree s;
    vector<int> size, id;

    heavy_light_decomposition(int r, vector<vector<int>> &g): n(g.size()), g(g), p(n), d(n)
, h(n), s(n), size(n), id(n) {
        d[r] = 0;
        dfs(r);
        h[r] = r;
        decompose(r);
    }

    void dfs(int a) {
        size[a] = 1;
        for (int &b : g[a]) if (b != p[a]) {
            p[b] = a;
            d[b] = d[a] + 1;
            dfs(b);
            size[a] += size[b];
            if (g[a][0] == p[a] or size[b] > size[g[a][0]]) {
                swap(g[a][0], b);
            }
        }
    }

    void decompose(int a) {
        id[a] = ++timer;
        for (int b : g[a]) if (b != p[a]) {
            if (b == g[a][0]) {
                h[b] = h[a];
            } else {
                h[b] = b;
            }
            decompose(b);
        }
    }

    int query(int a, int b) {
        int l = s.neutral;
        int r = s.neutral;
        while (h[a] != h[b]) {
            if (d[h[a]] > d[h[b]]) {
                l = s.combine(l, s.query(id[h[a]], id[a]));
                a = p[h[a]];
            } else {
                r = s.combine(s.query_reverse(id[h[b]], id[b]), r);
                b = p[h[b]];
            }
        }
        if (d[a] > d[b]) {
            l = s.combine(l, s.query(id[b], id[a]));
        } else {
            r = s.combine(s.query_reverse(id[a], id[b]), r);
        }
        return s.combine(l, r);
    }

    void modify(int a, int x) {
        s.modify(id[a], x);
    }
};
```

MATH

matrix.cpp

```
using matrix = vector<vector<int>>;

matrix operator*(const matrix &a, const matrix &b) {
    int n = a.size();
    int m = b[0].size();
    int h = b.size();
    matrix r(n, vector<int>(m));
    for (int i = 0; i < n; ++i) {
        for (int k = 0; k < h; ++k) {
            for (int j = 0; j < m; ++j) {
                r[i][j] = (r[i][j] + a[i][k] * b[k][j]) % mod;
            }
        }
    }
    return r;
}

matrix operator^(matrix a, int b) {
    int n = a.size();
    matrix r(n, vector<int>(n));
    for (int i = 0; i < n; ++i) r[i][i] = 1;
    while (b != 0) {
        if (b & 1) r = r * a;
        a = a * a;
        b >>= 1;
    }
    return r;
}
```

xor-basis.cpp

```
struct xor_basis {
    static constexpr int log = 30;
    int n = 0, s = 0;
    array<int, log> basis = {};

    void insert(int x) {
        for (int i = log - 1; i >= 0; --i) if ((x >> i) & 1) {
            if (basis[i] == 0) {
                for (int j = i - 1; j >= 0; --j) if ((x >> j) & 1) {
                    x ^= basis[j];
                }
                basis[i] = x;
                s += 1;
                for (int j = log - 1; j > i; --j) if ((basis[j] >> i) & 1) {
                    basis[j] ^= x;
                }
                break;
            } else {
                x ^= basis[i];
            }
        }
        n += 1;
    }

    bool contains(int x) {
        for (int i = log - 1; i >= 0; --i) if ((x >> i) & 1) {
            x ^= basis[i];
        }
        return x == 0;
    }

    int max() {
        int x = 0;
        for (int i = log - 1; i >= 0; --i) if (((x >> i) & 1) == 0) {
            x ^= basis[i];
        }
        return x;
    }

    int kth(int k) {
        if ((k >> s) != 0) return -1;
        int x = 0;
        for (int i = 0; i < log; ++i) if (basis[i] != 0) {
            if (k & 1) x ^= basis[i];
            k >>= 1;
        }
        return x;
    }

    int operator[](int k) {
        if (n - s > 60) return 0;
        return kth(k >> (n - s));
    }
};
```

MATH/CONVOLUTION

fast-fourier-transform.cpp

```cpp
const float PI = acos(-1);

void fast_fourier_transform(vector<complex<float>> &p, int sign) {
    int n = p.size();

    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        while (j & bit) j ^= bit, bit >>= 1;
        j ^= bit;
        if (i < j) swap(p[i], p[j]);
    }

    for (int len = 2; len <= n; len *= 2) {
        int half = len / 2;
        float ang = sign * (2 * PI / len);
        complex<float> step(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            complex<float> w(1, 0);
            for (int j = 0; j < half; ++j) {
                auto x = p[i + j];
                auto y = w * p[(i + j) + half];
                p[i + j] = x + y;
                p[(i + j) + half] = x - y;
                w *= step;
            }
        }
    }

    if (sign == -1) {
        for (complex<float> &x : p) x /= n;
    }
}

vector<int> convolution(vector<int> &a, vector<int> &b) {
    int n = a.size() + b.size() - 1;
    int m = 1;
    while (m < n) m *= 2;

    vector<complex<float>> fa(a.begin(), a.end());
    vector<complex<float>> fb(b.begin(), b.end());
    fa.resize(m);
    fb.resize(m);

    fast_fourier_transform(fa, +1);
    fast_fourier_transform(fb, +1);

    for (int i = 0; i < m; ++i) fa[i] *= fb[i];
    fast_fourier_transform(fa, -1);

    vector<int> c(n);
    for (int i = 0; i < n; ++i) {
        c[i] = round(fa[i].real());
    }
    return c;
}
```

fast-walsh-hadamard-transform.cpp

```cpp
void fast_walsh_hadamard_transform(vector<int> &p, int sign) {
    int n = p.size();
    for (int l = 1; l < n; l <<= 1) {
        for (int i = 0; i < n; i += l << 1) {
            for (int j = 0; j < l; ++j) {
                int x = p[i + j];
                int y = p[(i + j) + l];
                p[i + j] = x + y;
                p[(i + j) + l] = x - y;
            }
        }
    }

    if (sign == -1) {
        for (int &z : p) {
            z /= n;
        }
    }
}

vector<int> xor_convolution(vector<int> &a, vector<int> &b) {
    int m = max(a.size(), b.size());
    int n = (m == 1) ? 1 : 1 << (32 - __builtin_clz(m - 1));

    a.resize(n), fast_walsh_hadamard_transform(a, 1);
    b.resize(n), fast_walsh_hadamard_transform(b, 1);

    vector<int> c(n);
    for (int i = 0; i < n; ++i) c[i] = a[i] * b[i];
    fast_walsh_hadamard_transform(c, -1);
    return c;
}
```

fast-zeta-transform.cpp

```cpp
vector<int> or_convolution(vector<int> &a, vector<int> &b) {
    int s = max(a.size(), b.size());
    int m = (s == 1) ? 1 : 1 << (32 - __builtin_clz(s - 1));
    int n = __builtin_ctz(m);

    a.resize(m), b.resize(m);
    for (int i = 0; i < n; ++i) {
        for (int mask = 0; mask < m; ++mask) {
            if (mask & (1 << i)) {
                a[mask] += a[mask ^ (1 << i)];
                b[mask] += b[mask ^ (1 << i)];
            }
        }
    }

    vector<int> c(m);
    for (int mask = 0; mask < m; ++mask) {
        c[mask] = a[mask] * b[mask];
    }

    for (int i = 0; i < n; ++i) {
        for (int mask = 0; mask < m; ++mask) {
            if (mask & (1 << i)) {
                c[mask] -= c[mask ^ (1 << i)];
            }
        }
    }
    return c;
}

vector<int> and_convolution(vector<int> &a, vector<int> &b) {
    int s = max(a.size(), b.size());
    int m = (s == 1) ? 1 : 1 << (32 - __builtin_clz(s - 1));
    int n = __builtin_ctz(m);

    a.resize(m), b.resize(m);
    for (int i = 0; i < n; ++i) {
        for (int mask = 0; mask < m; ++mask) {
            if (not (mask & (1 << i))) {
                a[mask] += a[mask ^ (1 << i)];
                b[mask] += b[mask ^ (1 << i)];
            }
        }
    }

    vector<int> c(m);
    for (int mask = 0; mask < m; ++mask) {
        c[mask] = a[mask] * b[mask];
    }

    for (int i = 0; i < n; ++i) {
        for (int mask = 0; mask < m; ++mask) {
            if (not (mask & (1 << i))) {
                c[mask] -= c[mask ^ (1 << i)];
            }
        }
    }
    return c;
}
```

number-theoretic-transform.cpp

```cpp
const int mod = 998244353, g = 3;

void permute(vector<int> &p, vector<int> &order) {
    int n = p.size();
    for (int i = 0; i < n; ++i) {
        int j = order[i];
        if (i < j) swap(p[i], p[j]);
    }
}

void number_theoretic_transform(vector<int> &p, int sign, vector<int> &order, int inv) {
    int n = p.size();
    permute(p, order);

    for (int l = 1; l < n; l <<= 1) {
        int step = power(g, (mod - 1) / (l << 1));
        if (sign == -1) step = power(step, mod - 2);

        for (int i = 0; i < n; i += l << 1) {
            int w = 1;
            for (int j = 0; j < l; ++j) {
                int x = p[i + j];
                int y = (w * p[(i + j) + l]) % mod;
                p[i + j] = (x + y) % mod;
                p[(i + j) + l] = ((x - y) + mod) % mod;
                w = (w * step) % mod;
            }
        }
    }

    if (sign == -1) for (int &i : p) i = (i * inv) % mod;
}

vector<int> convolution(vector<int> &a, vector<int> &b) {
    int n = a.size() + b.size() - 1;
    int m = (n == 1) ? 1 : 1 << (32 - __builtin_clz(n - 1));
    int l = __builtin_ctz(m);

    vector<int> order(m);
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < l; ++j) {
            if (i & (1 << j)) {
                order[i] |= 1 << (l - (j + 1));
            }
        }
    }

    int inv = power(m, mod - 2);
    a.resize(m); number_theoretic_transform(a, 1, order, inv);
    b.resize(m); number_theoretic_transform(b, 1, order, inv);

    vector<int> c(m);
    for (int i = 0; i < m; ++i) c[i] = (a[i] * b[i]) % mod;
    number_theoretic_transform(c, -1, order, inv);
    c.resize(n);

    return c;
}
```

NUMBER-THEORY

mobius.cpp

```cpp
int mobius(int n) {
    int mu = 1;
    for (int i = 2; i * i <= n; ++i) if (n % i == 0) {
        n /= i;
        if (n % i == 0) return 0;
        mu *= -1;
    }
    if (n > 1) mu *= -1;
    return mu;
}

vector<int> mobius_sieve(int n) {
    vector<int> mu(n + 1);
    vector<bool> is_prime(n + 1, true);
    vector<int> primes;
    mu[1] = 1;
    is_prime[1] = false;
    for (int i = 2; i <= n; ++i) {
        if (is_prime[i]) {
            primes.push_back(i);
            mu[i] = -1;
        }
        for (int p : primes) {
            if (i * p > n) break;
            is_prime[i * p] = false;
            if (i % p == 0) {
                mu[i * p] = 0;
                break;
            } else {
                mu[i * p] = -mu[i];
            }
        }
    }
    return mu;
}
```

totient.cpp

```cpp
int totient(int n) {
    int phi = n;
    for (int i = 2; i * i <= n; ++i) if (n % i == 0) {
        phi -= phi / i;
        while (n % i == 0) n /= i;
    }
    if (n > 1) phi -= phi / n;
    return phi;
}

vector<int> totient_sieve(int n) {
    vector<int> phi(n + 1);
    for (int i = 1; i <= n; ++i) {
        phi[i] = i;
    }
    for (int i = 2; i <= n; ++i) if (phi[i] == i) {
        for (int j = i; j <= n; j += i) {
            phi[j] -= phi[j] / i;
        }
    }
    return phi;
}
```

NUMBER-THEORY/MOD

chinese-remainder-theorem.cpp

```cpp
pair<int, int> chinese_remainder_theorem(vector<int> &r, vector<int> &mod) {
    int n = mod.size();

    int i = r[0];
    int s = mod[0];
    for (int k = 1; k < n; ++k) {
        int j = r[k];
        int t = mod[k];

        int g = gcd(s, t);
        if (i % g != j % g) return {-1, -1};

        auto [_, x, y] = extended_gcd(s / g, t / g);
        int l = lcm(s, t);

        int a = ((int128(i) * (t / g) % l) * y) % l;
        int b = ((int128(j) * (s / g) % l) * x) % l;

        i = (((a + b) % l) + l) % l;
        s = l;
    }
    return {i, s};
}
```

extended-gcd.cpp

```cpp
tuple<int, int, int> extended_gcd(int a, int b) {
    if (b == 0) return {a, 1, 0};

    auto [g, x, y] = extended_gcd(b, a % b);
    return {g, y, x - (a / b) * y};
}
```

log.cpp

```cpp
int log(int a, int b, int mod) {
    if (a == 0) {
        if (b == 0) return 1 % mod;
        return -1;
    }

    int k = 1, h = 0;
    for (int g = gcd(a, mod); g != 1; g = gcd(a, mod), ++h) {
        if (k == b) return h;
        if (b % g) return -1;

        b /= g, mod /= g;
        k = (k * (a / g)) % mod;
    }

    int n = ceil(sqrt(mod)), s = 1;
    for (int i = 1; i <= n; ++i) {
        s = (s * a) % mod;
    }

    map<int, int> f;
    for (int q = 0; q <= n; ++q) {
        f[b] = q;
        b = (b * a) % mod;
    }

    a = (k * s) % mod;
    for (int p = 1; p <= n; ++p) {
        if (f.contains(a)) {
            return (n * p - f[a]) + h;
        }
        a = (a * s) % mod;
    }
    return -1;
}
```

multi-extended-gcd.cpp

```cpp
pair<int, vector<int>> multi_extended_gcd(vector<int> &a) {
    int n = a.size(), g = a[0];
    vector<int> c(n, 1), h(n, 1);

    for (int i = 1; i < n; ++i) {
        auto [f, x, y] = extended_gcd(g, a[i]);
        g = f;
        h[i - 1] = x;
        c[i] = y;
    }

    for (int i = n - 1; i >= 0; --i) {
        if (i < n - 1) h[i] *= h[i + 1];
        c[i] *= h[i];
    }
    return {g, c};
}
```

power.cpp

```cpp
int power(int128 a, int e, const int mod) {
    int128 r = 1;
    while (e != 0) {
        if (e & 1) r = (r * a) % mod;
        a = (a * a) % mod;
        e >>= 1;
    }
    return r;
}
```

NUMBER-THEORY/PRIME

linear-sieve.cpp

```cpp
vector<int> linear_sieve(int n) {
    vector<int> p, lp(n + 1);
    for (int i = 2; i <= n; ++i) {
        if (lp[i] == 0) {
            lp[i] = i;
            p.push_back(i);
        }
        for (int j : p) {
            if (j > lp[i] or i * j > n) break;
            lp[i * j] = j;
        }
    }
    return p;
}
```

miller-rabin.cpp

```cpp
bool composite(int n, int a, int s, int d) {
    int128 x = power(a, d, n);
    if (x == 1 or x == n - 1) return false;
    for (int i = 1; i < s; ++i) {
        x = (x * x) % n;
        if (x == n - 1) return false;
    }
    return true;
}

bool prime(int n) {
    if (n <= 1) return false;
    int s = 0, d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        s += 1;
    }
    for (int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (n == a) return true;
        if (composite(n, a, s, d)) return false;
    }
    return true;
}
```

segmented-sieve.cpp

```cpp
vector<int> segmented_sieve(int l, int r) {
    vector<int> primes = linear_sieve(sqrt(r));
    offset_array<char> is_prime(l, r + 1, true);
    for (int p : primes) {
        for (int i = max(p * ((l + p - 1) / p), p * p); i <= r; i += p) {
            is_prime[i] = false;
        }
    }
    primes.clear();
    for (int i = max(2, l); i <= r; ++i) {
        if (is_prime[i]) {
            primes.push_back(i);
        }
    }
    return primes;
}
```

SEARCH

ternary-search.cpp

```cpp
int ternary_search(int low, int high, bool max) {
    while (high - low > 3) {
        int mid_low = low + (high - low) / 3;
        int mid_high = high - (high - low) / 3;

        if (max == (value(mid_low) > value(mid_high))) {
            high = mid_high;
        } else {
            low = mid_low;
        }
    }

    int answer = low;
    for (int i = low + 1; i <= high; ++i) {
        if (max == (value(i) > value(answer))) {
            answer = i;
        }
    }
    return answer;
}

float ternary_search(float low, float high, bool max, float eps = 1e-6) {
    while (high - low > eps) {
        float mid_low = low + (high - low) / 3;
        float mid_high = high - (high - low) / 3;

        if (max == (value(mid_low) > value(mid_high))) {
            high = mid_high;
        } else {
            low = mid_low;
        }
    }
    return (low + high) / 2;
}
```

SQUARE-ROOT

decomposition.cpp

```cpp
struct decomposition {
    int n, m;
    vector<int> a, b;

    decomposition(vector<int> &base) {
        n = base.size();
        m = max<int>(1, sqrt(n));
        a.resize(n);
        b.resize((n + (m - 1)) / m);
        for (int i = 0; i < n; ++i) {
            a[i] = base[i];
            b[i / m] += a[i];
        }
    }

    void update(int i, int x) {
        a[i] = x;
        int b_i = i / m;
        b[b_i] = 0;
        int l = b_i * m;
        int r = min(n, (b_i + 1) * m) - 1;
        for (int k = l; k <= r; ++k) {
            b[b_i] += a[k];
        }
    }

    int query(int l, int r) {
        int s = 0;
        int b_l = l / m;
        int b_r = r / m;
        if (b_l == b_r) {
            for (int i = l; i <= r; ++i) {
                s += a[i];
            }
        } else {
            for (int i = l; i < (b_l + 1) * m; ++i) {
                s += a[i];
            }
            for (int i = b_l + 1; i < b_r; ++i) {
                s += b[i];
            }
            for (int i = b_r * m; i <= r; ++i) {
                s += a[i];
            }
        }
        return s;
    }
};
```

mo.cpp

```cpp
int hilbert(int l, int r) {
    static array<int, 4> d = {3, 0, 0, 1};
    int h = 0, c = 0;
    for (int s = 20; s >= 0; --s) {
        int q = (((l >> s & 1) << 1 | (r >> s & 1)) + c) & 3;
        h = h << 2 | q;
        c = (c + d[q]) & 3;
    }
    return h;
}

vector<int> mo(vector<int> &a, vector<array<int, 4>> &q) {
    for (auto &[h, l, r, id] : q) h = hilbert(l, r);
    sort(q.begin(), q.end());

    auto insert = [&](int x) -> void {};
    auto remove = [&](int x) -> void {};

    vector<int> ans(q.size());
    int t = 0, i = 0, j = -1;
    for (auto &[h, l, r, id] : q) {
        while (i > l) insert(a[--i]);
        while (j < r) insert(a[++j]);
        while (i < l) remove(a[i++]);
        while (j > r) remove(a[j--]);
        ans[id] = t;
    }
    return ans;
}
```

STRING

aho-corasick.cpp

```cpp
struct aho_corasick {
private:
    struct node {
        int fail = 0;
        int exit = 0;
        array<int, 26> next = {};
        vector<int> end;
    };

    vector<node> trie;

public:
    aho_corasick(int n = 0) {
        trie.reserve(n + 5);
        trie.emplace_back();
    }

    void insert(string &s) {
        int i = 0;
        for (char c : s) {
            int j = c - 'a';
            if (trie[i].next[j] == 0) {
                trie[i].next[j] = trie.size();
                trie.emplace_back();
            }
            i = trie[i].next[j];
        }
        trie[i].end.push_back(s.size());
    }

    void build() {
        queue<int> q;
        for (int j = 0; j < 26; ++j) {
            int i = trie[0].next[j];
            if (i != 0) {
                q.push(i);
            }
        }
        while (not q.empty()) {
            int i = q.front(); q.pop();
            int f = trie[i].fail;
            if (not trie[f].end.empty()) {
                trie[i].exit = f;
            } else {
                trie[i].exit = trie[f].exit;
            }
            for (int j = 0; j < 26; ++j) {
                int c = trie[i].next[j];
                if (c != 0) {
                    trie[c].fail = trie[f].next[j];
                    q.push(c);
                } else {
                    trie[i].next[j] = trie[f].next[j];
                }
            }
        }
    }

    vector<int> match(string &t) {
        vector<int> p;
        int n = t.length();
        for (int i = 0, k = 0; k < n; ++k) {
            i = trie[i].next[t[k] - 'a'];
            for (int j = i; j != 0; j = trie[j].exit) {
                for (int l : trie[j].end) {
                    p.push_back(k - l + 1);
                }
            }
        }
        return p;
    }
};
```

hasher-segment-tree.cpp

```cpp
struct hasher_segment_tree {
private:
    static constexpr array<int, 2> base = {41, 53};
    static constexpr array<int, 2> mod = {1000000007, 1000000009};

    struct node {
        int len = 0;
        array<int, 2> f_hash = {};
        array<int, 2> r_hash = {};

        node() = default;

        node(char c): len(1) {
            f_hash[0] = r_hash[0] = c % mod[0];
            f_hash[1] = r_hash[1] = c % mod[1];
        }
    };

    int n;
    vector<node> tree;
    vector<array<int, 2>> pow;

    node merge(const node &l, const node &r) {
        if (l.len == 0) return r;
        if (r.len == 0) return l;
        node m;
        m.len = l.len + r.len;
        m.f_hash[0] = (l.f_hash[0] * pow[r.len][0] + r.f_hash[0]) % mod[0];
        m.r_hash[0] = (r.r_hash[0] * pow[l.len][0] + l.r_hash[0]) % mod[0];
        m.f_hash[1] = (l.f_hash[1] * pow[r.len][1] + r.f_hash[1]) % mod[1];
        m.r_hash[1] = (r.r_hash[1] * pow[l.len][1] + l.r_hash[1]) % mod[1];
        return m;
    }

public:
    hasher_segment_tree(const string &s): n(s.length()), tree(2 * n), pow(n) {
        pow[0][0] = pow[0][1] = 1;
        for (int i = 1; i < n; ++i) {
            pow[i][0] = (pow[i - 1][0] * base[0]) % mod[0];
            pow[i][1] = (pow[i - 1][1] * base[1]) % mod[1];
        }

        for (int i = 0; i < n; ++i) tree[n + i] = node(s[i]);
        for (int i = n - 1; i > 0; --i) tree[i] = merge(tree[i << 1], tree[(i << 1) | 1]);
    }

    void modify(int i, char c) {
        i += n;
        tree[i] = node(c);
        for (i >>= 1; i > 0; i >>= 1) tree[i] = merge(tree[i << 1], tree[(i << 1) | 1]);
    }

    node query(int i, int j) {
        node l, r;
        for (i += n, j += n + 1; i < j; i >>= 1, j >>= 1) {
            if (i & 1) l = merge(l, tree[i++]);
            if (j & 1) r = merge(tree[--j], r);
        }
        return merge(l, r);
    }

    bool palindrome(int i, int j) {
        if (i >= j) return true;
        node q = query(i, j);
        return q.f_hash == q.r_hash;
    }
};
```

hasher.cpp

```cpp
const array<int, 2> base = {41, 53};
const array<int, 2> mod = {1000000007, 1000000009};

struct hasher {
    int n;
    vector<array<int, 2>> pow;
    vector<array<int, 2>> pref, suf;

    hasher(const string &s): n(s.length()), pow(n + 1), pref(n + 1), suf(n + 2) {
        pow[0] = {1, 1};
        for (int i = 0; i < n; ++i) for (int j : {0, 1}) {
            pow[i + 1][j] = (pow[i][j] * base[j]) % mod[j];
            pref[i + 1][j] = (pref[i][j] * base[j] + s[i]) % mod[j];
            suf[n - i][j] = (suf[n - i + 1][j] * base[j] + s[n - i - 1]) % mod[j];
        }
    }

    int get(vector<array<int, 2>> &h, int l, int r) {
        array<int, 2> a;
        for (int i : {0, 1}) {
            a[i] = (h[r][i] - h[l][i] * pow[abs(r - l)][i]) % mod[i];
            if (a[i] < 0) a[i] += mod[i];
        }
        return (a[0] << 32) | a[1];
    }

    int normal(int l, int r) {
        return get(pref, l, r + 1);
    }
```

```
    int reverse(int l, int r) {
        return get(suf, r + 2, l + 1);
    }
};
```

longest-prefix-suffix.cpp

```
vector<int> longest_prefix_suffix(string &s) {
    int n = s.length();
    vector<int> lps(n);
    for (int i = 1; i < n; ++i) {
        int j = lps[i - 1];
        while (j != 0 and s[i] != s[j]) {
            j = lps[j - 1];
        }
        if (s[i] == s[j]) {
            j++;
        }
        lps[i] = j;
    }
    return lps;
}
```

manacher.cpp

```
string pre_process(string &s) {
    string t = "#";
    for (char &c : s) {
        t.push_back(c);
        t.push_back('#');
    }
    return t;
}

vector<int> manacher(string &s) {
    string t = '^' + pre_process(s) + '$';
    int n = t.length();

    vector<int> p(n);
    int c = 0, r = 0;
    for (int i = 1; i < n - 1; i++) {
        int j = 2 * c - i;
        if (i < r) {
            p[i] = min(r - i, p[j]);
        }
        while (t[i + (p[i] + 1)] == t[i - (p[i] + 1)]) {
            p[i]++;
        }
        if (i + p[i] > r) {
            c = i;
            r = i + p[i];
        }
    }
    return p;
}

string longest_palindrome(string &s) {
    vector<int> p = manacher(s);
    int n = p.size();

    int k = 0, l = 0;
    for (int i = 1; i < n - 1; i++) {
        if (p[i] > l) {
            l = p[i];
            k = (i - l) / 2;
        }
    }
    return s.substr(k, l);
}

int count_palindromes(string &s) {
    vector<int> p = manacher(s);
    int n = p.size();

    int total = 0;
    for (int i = 1; i < n - 1; ++i) {
        total += (p[i] + 1) / 2;
    }
    return total;
}
```

palindromic-tree.cpp

```
struct palindromic_tree {
    struct node {
        int length, fail;
        array<int, 26> child = {};

        node(int length): length(length), fail(0) {}
    };

    int suffix;
    vector<char> text;
    vector<node> tree;

    palindromic_tree(int n): suffix(0) {
        text.reserve(n + 5);
        text.push_back('$');
        tree.reserve(n + 5);
        tree.push_back(node(0));
        tree.push_back(node(-1));
        tree[0].fail = 1;
    }

    int follow(int id) {
        while (text[(text.size() - tree[id].length) - 2] != text.back()) {
            id = tree[id].fail;
        }
        return id;
    }

    void insert(char c) {
        text.push_back(c);
        suffix = follow(suffix);

        if (not tree[suffix].child[c - 'a']) {
            int id = tree.size();
            tree.push_back(node(tree[suffix].length + 2));

            tree[id].fail = tree[follow(tree[suffix].fail)].child[c - 'a'];
            tree[suffix].child[c - 'a'] = id;
        }
        suffix = tree[suffix].child[c - 'a'];
    }
};
```

suffix-automaton.cpp

```
struct suffix_automaton {
    struct node {
        int len;
        int link;
        array<int, 26> next;

        node(int len): len(len) {
            link = -1;
            next.fill(-1);
        }
    };

    vector<node> states;
    int last;

    suffix_automaton(int n = 1) {
        states.reserve(2 * n);
        states.emplace_back(0);
        last = 0;
    }

    void extend(char c) {
        int curr = states.size();
        states.emplace_back(states[last].len + 1);
        int p = last;
        while (p != -1 and states[p].next[c - 'a'] == -1) {
            states[p].next[c - 'a'] = curr;
            p = states[p].link;
        }
        if (p == -1) {
            states[curr].link = 0;
        } else {
            int q = states[p].next[c - 'a'];
            if (states[q].len == states[p].len + 1) {
                states[curr].link = q;
            } else {
                int clone = states.size();
                states.push_back(states[q]);
                states[clone].len = states[p].len + 1;
                while (p != -1 and states[p].next[c - 'a'] == q) {
                    states[p].next[c - 'a'] = clone;
                    p = states[p].link;
                }
                states[q].link = clone;
                states[curr].link = clone;
            }
        }
        last = curr;
    }
};
```

trie.cpp

```
struct trie {
private:
    struct node {
        int pref = 0;
        int end = 0;
        array<int, 26> child = {};
    };

    vector<node> tree;

    void dfs(int i, string &s, vector<string> &r) {
        if (tree[i].end != 0) {
            r.push_back(s);
        }
        for (int j = 0; j < 26; ++j) {
            if (tree[i].child[j]) {
                s.push_back(j + 'a');
                dfs(tree[i].child[j], s, r);
                s.pop_back();
            }
        }
    }

public:
    trie() {
        tree.emplace_back();
    }

    void insert(string &s) {
        int i = 0;
        tree[0].pref++;
        for (char c : s) {
            int j = c - 'a';
            if (tree[i].child[j] == 0) {
                tree[i].child[j] = tree.size();
                tree.emplace_back();
            }
            i = tree[i].child[j];
            tree[i].pref++;
        }
        tree[i].end++;
    }

    void remove(string &s) {
        int i = 0;
        for (char c : s) {
            int j = c - 'a';
            if (tree[i].child[j] == 0) {
                return;
            }
            i = tree[i].child[j];
        }
        if (tree[i].end == 0) return;
        int p = 0;
        tree[0].pref--;
        for (char c : s) {
            int j = c - 'a';
            i = tree[p].child[j];
            tree[i].pref--;
            if (tree[i].pref == 0) {
                tree[p].child[j] = 0;
                return;
            }
            p = i;
        }
        tree[p].end--;
    }

    bool search(string &s) {
        int i = 0;
        for (char c : s) {
            int j = c - 'a';
            if (tree[i].child[j] == 0) {
                return false;
            }
            i = tree[i].child[j];
        }
        return tree[i].end != 0;
    }
```

```
    vector<string> complete(string s) {
        int i = 0;
        vector<string> r;
        for (char c : s) {
            int j = c - 'a';
            if (tree[i].child[j] == 0) {
                return r;
            }
            i = tree[i].child[j];
        }
        dfs(i, s, r);
        return r;
    }
};
```

z-array.cpp

```
vector<int> z_array(string &s) {
    int n = s.length();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        z[i] = max(0, min(z[i - l], r - (i + 1)));
        while (i + z[i] < n and s[i + z[i]] == s[z[i]]) {
            l = i;
            r = i + z[i];
            z[i] += 1;
        }
    }
    return z;
}
```

STRUCTURE

coprime-set.cpp

```
struct coprime_set {
    map<int, int> n;

    vector<pair<int, int>> divisors(vector<int> &p) {
        int k = p.size();
        vector<pair<int, int>> a; a.reserve(1 << k);
        for (int m = 0; m < (1 << k); ++m) {
            int d = 1, mu = 1;
            for (int i = 0; i < k; ++i) if (m & (1 << i)) {
                d *= p[i], mu *= -1;
            }
            a.push_back({d, mu});
        }
        return a;
    }

    void insert(vector<int> &p) {
        vector<pair<int, int>> a = divisors(p);
        for (auto &[d, mu] : a) {
            n[d]++;
        }
    }

    void remove(vector<int> &p) {
        vector<pair<int, int>> a = divisors(p);
        for (auto &[d, mu] : a) {
            n[d]--;
        }
    }

    int query(vector<int> &p) {
        int t = 0;
        vector<pair<int, int>> a = divisors(p);
        for (auto &[d, mu] : a) {
            t += n[d] * mu;
        }
        return t;
    }
};
```

cover-tree.cpp

```
struct cover_tree {
    int n;
    vector<int> cnt, len, all;

    void recalc(int i) {
        if (cnt[i] > 0) len[i] = all[i];
        else if (i < n) len[i] = len[i << 1] + len[i << 1 | 1];
        else len[i] = 0;
    }

    cover_tree(vector<int> &x): n(x.size() - 1), cnt(2 * n), len(2 * n), all(2 * n) {
        for (int i = 0; i < n; ++i) all[n + i] = x[i + 1] - x[i];
        for (int i = n - 1; i > 0; --i) all[i] = all[i << 1] + all[i << 1 | 1];
    }

    void update(int i, int j, int x) {
        int p = i, q = j;
        for (i += n, j += n; i < j; i >>= 1, j >>= 1) {
            if (i & 1) cnt[i] += x, recalc(i++);
            if (j & 1) cnt[--j] += x, recalc(j);
        }
        i = p + n;
        while (i >>= 1) recalc(i);
        j = q + n - 1;
        while (j >>= 1) recalc(j);
    }

    int query() {
        return len[1];
    }
};
```

disjoint-set-union.cpp

```
struct disjoint_set_union {
    vector<int> p, s;

    disjoint_set_union(int n): p(n), s(n, 1) {
        for (int i = 0; i < n; ++i) p[i] = i;
    }

    int find(int a) {
        if (p[a] == a) return a;
        return p[a] = find(p[a]);
    }

    void unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return;
        if (s[a] < s[b]) swap(a, b);
        p[b] = a;
        s[a] += s[b];
    }
};
```

line-container.cpp

```
struct line {
    int a, b;
    mutable int i;

    bool operator<(const line &other) const {
        return tie(a, b) < tie(other.a, other.b);
    }

    bool operator<(const int x) const {
        return i < x;
    }
};

struct line_container : multiset<line, less<>> {
private:
    int intersect(iterator s, iterator t) {
        int x = t->b - s->b, y = s->a - t->a;
        return x / y - ((x ^ y) < 0 and x % y != 0);
    }

    void update(iterator s) {
        if (next(s) == end()) {
            s->i = inf;
        } else {
            s->i = intersect(s, next(s));
        }
    }

public:
    void insert(int a, int b) {
        iterator l = multiset::insert({a, b, 0});

        if (l != begin() and prev(l)->a == l->a) {
            erase(prev(l));
        }
        if (next(l) != end() and next(l)->a == l->a) {
            erase(l);
            return;
        }

        if (l != begin() and next(l) != end()) {
            iterator p = prev(l), n = next(l);
            if (intersect(p, l) >= intersect(l, n)) {
                erase(l);
                return;
            }
        }

        while (l != begin()) {
            iterator p = prev(l);
            if (p != begin() and intersect(prev(p), p) >= intersect(p, l)) {
                erase(p);
                continue;
            }
            break;
        }
        while (next(l) != end()) {
            iterator n = next(l);
            if (next(n) != end() and intersect(l, n) >= intersect(n, next(n))) {
                erase(n);
                continue;
            }
            break;
        }

        if (l != begin()) {
            update(prev(l));
        }
        update(l);
    }

    int query(int x) {
        auto l = lower_bound(x);
        return l->a * x + l->b;
    }
};
```

merge-sort-tree.cpp

```
struct merge_sort_tree {
    int n;
    vector<int> a;
    vector<ordered_set<pair<int, int>>> tree;

    merge_sort_tree(vector<int> &a): n(a.size()), a(a), tree(2 * n) {
        for (int i = 0; i < n; ++i) {
            for (int j = i + n; j > 0; j >>= 1) {
                tree[j].insert({a[i], i});
            }
        }
    }

    void update(int i, int x) {
        for (int j = i + n; j > 0; j >>= 1) {
            tree[j].erase({a[i], i});
            tree[j].insert({x, i});
        }
        a[i] = x;
    }

    int query(int i, int j, int k) {
        int c = 0;
        for (i += n, j += n + 1; i < j; i >>= 1, j >>= 1) {
            if (i & 1) c += tree[i++].order_of_key({k + 1, 0});
            if (j & 1) c += tree[--j].order_of_key({k + 1, 0});
        }
        return c;
    }
};
```

range-set.cpp

```
struct range_set {
    static constexpr int size = 1024;
    static constexpr int n = (size + 63) / 64;

    uint word[n] = {};

    bool contains(int x) {
```

```
        return word[x >> 6] & (1ULL << (x & 63));
    }

    void insert(int x) {
        word[x >> 6] |= 1ULL << (x & 63);
    }

    void remove(int x) {
        word[x >> 6] &= ~(1ULL << (x & 63));
    }

    bool empty() {
        for (int i = 0; i < n; ++i) {
            if (word[i]) return false;
        }
        return true;
    }

    void insert(int l, int r) {
        int i = l >> 6;
        int j = r >> 6;
        if (i == j) {
            word[i] |= (~0ULL << (l & 63)) & (~0ULL >> (63 - (r & 63)));
        } else {
            word[i] |= ~0ULL << (l & 63);
            for (int k = i + 1; k < j; ++k) {
                word[k] = ~0ULL;
            }
            word[j] |= ~0ULL >> (63 - (r & 63));
        }
    }

    void remove(int l, int r) {
        int i = l >> 6;
        int j = r >> 6;
        if (i == j) {
            word[i] &= ~((~0ULL << (l & 63)) & (~0ULL >> (63 - (r & 63))));
        } else {
            word[i] &= ~(~0ULL << (l & 63));
            for (int k = i + 1; k < j; ++k) {
                word[k] = 0;
            }
            word[j] &= ~(~0ULL >> (63 - (r & 63)));
        }
    }

    int prev(int x) {
        int i = x >> 6;
        uint curr = word[i] & (~0ULL >> (63 - (x & 63)));
        if (curr) {
            return (i << 6) + (63 - __builtin_clzll(curr));
        } else {
            for (int k = i - 1; k >= 0; --k) if (word[k]) {
                return (k << 6) + (63 - __builtin_clzll(word[k]));
            }
        }
        return -1;
    }

    int next(int x) {
        int i = x >> 6;
        uint curr = word[i] & (~0ULL << (x & 63));
        if (curr) {
            return (i << 6) + __builtin_ctzll(curr);
        } else {
            for (int k = i + 1; k < n; ++k) if (word[k]) {
                return (k << 6) + __builtin_ctzll(word[k]);
            }
        }
        return size;
    }

    int segments() {
        int total = 0;
        for (int i = 0; i < n; ++i) {
            int count = __builtin_popcountll((word[i] >> 1) & ~word[i]);
            if ((word[i] & 1ULL) and (i == 0 or not (word[i - 1] & (1ULL << 63)))) count +=
1;
            total += count;
        }
        return total;
    }
};
```

sparse-table.cpp

```
struct sparse_table {
    int n;
    vector<vector<int>> table;

    int merge(int x, int y) {
        return gcd(x, y);
    }

    sparse_table(vector<int> &a): n(a.size()), table(__lg(n) + 1, vector<int>(n)) {
        for (int i = 0; i < n; ++i) table[0][i] = a[i];
        for (int i = 1; i <= __lg(n); ++i) {
            for (int j = 0; j + (1 << i) <= n; ++j) {
                table[i][j] = merge(table[i - 1][j], table[i - 1][j + (1 << (i - 1))]);
            }
        }
    }

    int query(int i, int j) {
        int k = __lg(j - i + 1);
        return merge(table[k][i], table[k][j - (1 << k) + 1]);
    }
};
```

treap.cpp

```
struct treap {
private:
    struct node {
        int x = 0, y = 0;
        int l = 0, r = 0;
        int s = 1;
    };

    int make(int x, int y) {
        int i = tree.size();
        tree.emplace_back();
        tree[i].x = x;
        tree[i].y = y;
        return i;
    }

    int size(int i) {
        return i ? tree[i].s : 0;
    }

    void update(int i) {
        tree[i].s = 1 + size(tree[i].l) + size(tree[i].r);
    }

    pair<int, int> split(int i, int x) {
        if (not i) return {0, 0};

        if (tree[i].x <= x) {
            auto [l, r] = split(tree[i].r, x);
            tree[i].r = l;
            update(i);
            return {i, r};
        } else {
            auto [l, r] = split(tree[i].l, x);
            tree[i].l = r;
            update(i);
            return {l, i};
        }
    }

    int merge(int l, int r) {
        if (not l) return r;
        if (not r) return l;

        if (tree[l].y > tree[r].y) {
            tree[l].r = merge(tree[l].r, r);
            update(l);
            return l;
        } else {
            tree[r].l = merge(l, tree[r].l);
            update(r);
            return r;
        }
    }

    int insert(int i, int x, int y) {
        if (not i) return make(x, y);

        if (tree[i].y < y) {
            auto [l, r] = split(i, x);
            i = make(x, y);
            tree[i].l = l;
            tree[i].r = r;
        } else {
            if (x <= tree[i].x) tree[i].l = insert(tree[i].l, x, y);
            if (x > tree[i].x) tree[i].r = insert(tree[i].r, x, y);
        }
        update(i);
        return i;
    }

    int erase(int i, int x) {
        if (not i) return 0;

        if (x == tree[i].x) return merge(tree[i].l, tree[i].r);
        if (x < tree[i].x) tree[i].l = erase(tree[i].l, x);
        if (x > tree[i].x) tree[i].r = erase(tree[i].r, x);
        update(i);
        return i;
    }

    int index(int i, int x) {
        if (not i) return -1;

        if (x == tree[i].x) return size(tree[i].l);
        if (x < tree[i].x) return index(tree[i].l, x);

        int r = index(tree[i].r, x);
        if (r == -1) return -1;
        return 1 + size(tree[i].l) + r;
    }

    int kth(int i, int x) {
        if (not i) return -1;

        int l = size(tree[i].l);
        if (x == l) return tree[i].x;

        if (x < l) return kth(tree[i].l, x);
        return kth(tree[i].r, x - (l + 1));
    }

public:
    vector<node> tree;
    int root;

    treap() {
        root = make(0, 0);
    }

    void insert(int x) {
        int y = random;
        root = insert(root, x, y);
    }

    void erase(int x) {
        root = erase(root, x);
    }

    int index(int x) {
        return index(root, x);
    }

    int kth(int i) {
        return kth(root, i);
    }

    int unite(int i, int j) {
        if (not i) return j;
        if (not j) return i;

        if (tree[i].y < tree[j].y) swap(i, j);
        auto [l, r] = split(j, tree[i].x);

        tree[i].l = unite(tree[i].l, l);
        tree[i].r = unite(tree[i].r, r);
        update(i);
        return i;
    }
};
```

venice-set.cpp

```
struct venice_set {
    int k = 0;
    multiset<int> s;

    int size() {
        return s.size();
    }

    void insert(int x) {
        s.insert(x - k);
    }

    void remove(int x) {
```

```
        if (auto i = s.find(x - k); i != s.end()) s.erase(i);
    }

    void update(int x) {
        k += x;
    }

    int min() {
        return *s.begin() + k;
    }
};
```

wavelet-tree.cpp

```
struct wavelet_tree {
private:
    struct node {
        int l, r, low, high;
        vector<int> pref;

        node(int low = 0, int high = 0, int size = 0): l(0), r(0), low(low), high(high), pr
ef(size + 1) {}
    };

    int root;
    vector<node> tree;

    int build(vector<int> &a, int low, int high) {
        int p = tree.size();
        tree.push_back(node(low, high, a.size()));

        if (low == high) {
            for (int i = 0; i < a.size(); ++i) {
                tree[p].pref[i + 1] = tree[p].pref[i] + 1;
            }
        } else {
            int mid = (low + high) / 2;
            vector<int> l; l.reserve(a.size());
            vector<int> r; r.reserve(a.size());

            for (int i = 0; i < a.size(); ++i) {
                tree[p].pref[i + 1] = tree[p].pref[i];
                if (a[i] <= mid) {
                    tree[p].pref[i + 1] += 1;
                    l.push_back(a[i]);
                } else {
                    r.push_back(a[i]);
                }
            }

            if (l.size()) tree[p].l = build(l, low, mid);
            if (r.size()) tree[p].r = build(r, mid + 1, high);
        }
        return p;
    }

    int count_less_equal(int ql, int qr, int x, int p) {
        if (not p or (tree[p].low > x or ql > qr)) return 0;

        if (tree[p].high <= x) return (qr - ql) + 1;

        int prev = tree[p].pref[ql];
        int curr = tree[p].pref[qr + 1] - tree[p].pref[ql];

        return count_less_equal(prev, (curr + prev) - 1, x, tree[p].l) + count_less_equal(q
l - prev, qr - (prev + curr), x, tree[p].r);
    }

    int kth(int ql, int qr, int k, int p) {
        if (tree[p].low == tree[p].high) return tree[p].low;

        int prev = tree[p].pref[ql];
        int curr = tree[p].pref[qr + 1] - tree[p].pref[ql];

        if (k < curr) {
            return kth(prev, (curr + prev) - 1, k, tree[p].l);
        } else {
            return kth(ql - prev, qr - (curr + prev), k - curr, tree[p].r);
        }
    }

public:
    wavelet_tree(vector<int> &a): tree(1) {
        tree.reserve(2 * a.size());
        root = build(a, *min_element(a.begin(), a.end()), *max_element(a.begin(), a.end()))
;
    }

    int count_less_equal(int l, int r, int x) {
        return count_less_equal(l, r, x, root);
    }

    int kth(int l, int r, int k) {
        return kth(l, r, k, root);
    }
};
```

STRUCTURE/BINARY-INDEXED-TREE

binary-indexed-tree-2d.cpp

```
struct binary_indexed_tree_2d {
    int n, m;
    vector<binary_indexed_tree> tree;

    binary_indexed_tree_2d(int n = 0, int m = 0): n(n), m(m), tree(n + 1, binary_indexed_tr
ee(m)) {}

    void build(vector<vector<int>> &a) {
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                update(i, j, a[i][j]);
            }
        }
    }

    void update(int i, int j, int x) {
        for (i += 1; i <= n; i += i & -i) tree[i].update(x);
    }

    int prefix(int i, int j) {
        int s = 0;
        for (i += 1; i > 0; i -= i & -i) s += tree[i].prefix(j);
        return s;
    }

    int query(int i, int j, int p, int q) {
        return prefix(p, q)
            - prefix(i - 1, q)
            - prefix(p, j - 1)
            + prefix(i - 1, j - 1);
    }
};
```

binary-indexed-tree.cpp

```
struct binary_indexed_tree {
    int n;
    vector<int> tree;

    binary_indexed_tree(int n = 0): n(n), tree(n + 1) {}

    void update(int i, int x) {
        for (i += 1; i <= n; i += i & -i) tree[i] += x;
    }

    int prefix(int i) {
        int s = 0;
        for (i += 1; i > 0; i -= i & -i) s += tree[i];
        return s;
    }

    int query(int i, int j) {
        return prefix(j) - prefix(i - 1);
    }
};
```

sparse-binary-indexed-tree.cpp

```
struct sparse_binary_indexed_tree {
    int n;
    vector<int> is;
    vector<vector<int>> js;
    vector<binary_indexed_tree> tree;

    void compress(vector<int> &a) {
        sort(a.begin(), a.end());
        a.erase(unique(a.begin(), a.end()), a.end());
    }

    int id(vector<int> &a, int i) {
        return lower_bound(a.begin(), a.end(), i) - a.begin();
    }

    sparse_binary_indexed_tree(vector<tuple<char, int, int>> &q) {
        for (auto [t, i, j] : q) is.push_back(i);
        compress(is);
        n = is.size();

        js.resize(n + 1);
        for (auto [t, i, j] : q) {
            if (t == 'u') for (i = id(is, i) + 1; i <= n; i += i & -i) js[i].push_back(j);
            else if (t == 'p') for (i = id(is, i) + 1; i > 0; i -= i & -i) js[i].push_back(
j);
        }

        tree.resize(n + 1);
        for (int i = 1; i <= n; ++i) {
            compress(js[i]);
            tree[i] = binary_indexed_tree(js[i].size());
        }
    }

    void update(int i, int j, int x) {
        for (i = id(is, i) + 1; i <= n; i += i & -i) tree[i].update(id(js[i], j), x);
    }

    int prefix(int i, int j) {
        int s = 0;
        for (i = id(is, i) + 1; i > 0; i -= i & -i) s += tree[i].prefix(id(js[i], j));
        return s;
    }
};
```

STRUCTURE/POLICY-BASED-DATA-STRUCTURES

hash-table.cpp

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

struct custom_hash {
    size_t operator()(size_t x) const {
        static const size_t time = chrono::steady_clock::now().time_since_epoch().count();
        return (x + 0x9e3779b9) ^ time;
    }
};

template<typename T>
using hash_set = gp_hash_table<T, null_type, custom_hash>;

template<typename K, typename V>
using hash_map = gp_hash_table<K, V, custom_hash>;
```

ordered-set.cpp

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_upd
ate>;
```

STRUCTURE/SEGMENT-TREE

lazy-segment-tree.cpp

```
struct lazy_segment_tree {
    int n, h;
    vector<int> tree, lazy, len;

    int merge(int x, int y) {
        return x + y;
    }

    void apply(int i, int x) {
        tree[i] += x * len[i];
        if (i < n) lazy[i] += x;
```

```cpp
    }

    void push(int k) {
        for (int s = h; s > 0; --s) {
            int i = k >> s;
            if (lazy[i] == 0) continue;
            apply(i << 1, lazy[i]);
            apply((i << 1) | 1, lazy[i]);
            lazy[i] = 0;
        }
    }

    void pull(int i) {
        while (i >>= 1) {
            tree[i] = merge(tree[i << 1], tree[(i << 1) | 1]);
            if (lazy[i] != 0) tree[i] += lazy[i] * len[i];
        }
    }

    lazy_segment_tree(int n): n(n), h(__lg(2 * n - 1)), tree(2 * n), lazy(n), len(2 * n) {
        for (int i = n; i < 2 * n; ++i) len[i] = 1;
        for (int i = n - 1; i > 0; --i) len[i] = len[i << 1] + len[(i << 1) | 1];
    }

    lazy_segment_tree(vector<int> &a): lazy_segment_tree(a.size()) {
        for (int i = 0; i < n; ++i) tree[n + i] = a[i];
        for (int i = n - 1; i > 0; --i) tree[i] = merge(tree[i << 1], tree[(i << 1) | 1]);
    }

    void update(int i, int j, int x) {
        int p = i, q = j;
        push(p + n), push(q + n);
        for (i += n, j += n + 1; i < j; i >>= 1, j >>= 1) {
            if (i & 1) apply(i++, x);
            if (j & 1) apply(--j, x);
        }
        pull(p + n), pull(q + n);
    }

    int query(int i, int j) {
        push(i + n), push(j + n);
        int l = 0, r = 0;
        for (i += n, j += n + 1; i < j; i >>= 1, j >>= 1) {
            if (i & 1) l = merge(l, tree[i++]);
            if (j & 1) r = merge(tree[--j], r);
        }
        return merge(l, r);
    }
};
```

persistent-segment-tree.cpp

```cpp
struct persistent_segment_tree {
private:
    struct node {
        int x, l, r;

        node(int x = 0, int l = 0, int r = 0): x(x), l(l), r(r) {}
    };

    int n;
    vector<node> tree;

    int terminal(int x) {
        int i = tree.size();
        tree.push_back(node(x));
        return i;
    }

    int internal(int l, int r) {
        int i = tree.size();
        tree.push_back(node(tree[l].x + tree[r].x, l, r));
        return i;
    }

    int build(vector<int> &a, int l, int r) {
        if (l == r) return terminal(a[l]);

        int m = (l + r) / 2;
        return internal(build(a, l, m), build(a, m + 1, r));
    }

    int modify(int i, int x, int p, int l, int r) {
        if (l == r) return terminal(tree[p].x + x);

        int m = (l + r) / 2;
        if (i <= m) return internal(modify(i, x, tree[p].l, l, m), tree[p].r);
        else return internal(tree[p].l, modify(i, x, tree[p].r, m + 1, r));
    }

    int query(int ql, int qr, int p, int l, int r) {
        if (not p or (ql > r or qr < l)) return 0;

        if (ql <= l and qr >= r) return tree[p].x;

        int m = (l + r) / 2;
        return query(ql, qr, tree[p].l, l, m) + query(ql, qr, tree[p].r, m + 1, r);
    }

    int copy(int ql, int qr, int p, int q, int l, int r) {
        if (ql > r or qr < l) return p;

        if (ql <= l and qr >= r) return q;

        int m = (l + r) / 2;
        return internal(copy(ql, qr, tree[p].l, tree[q].l, l, m), copy(ql, qr, tree[p].r, t
ree[q].r, m + 1, r));
    }

public:
    vector<int> root;

    persistent_segment_tree(int n): n(n), tree(1) {
        root.reserve(n);
        tree.reserve(8 * n);
    }

    void build(vector<int> &a) {
        root.push_back(build(a, 0, n - 1));
    }

    void modify(int i, int x, int p) {
        root.push_back(modify(i, x, p, 0, n - 1));
    }

    int query(int l, int r, int p) {
        return query(l, r, p, 0, n - 1);
    }

    void copy(int l, int r, int p, int q) {
        root.push_back(copy(l, r, p, q, 0, n - 1));
    }
};
```

segment-tree.cpp

```cpp
struct segment_tree {
    int n;
    vector<int> tree;

    int merge(int x, int y) {
        return x + y;
    }

    segment_tree(int n): n(n), tree(2 * n) {}

    segment_tree(vector<int> &a): segment_tree(a.size()) {
        for (int i = 0; i < n; ++i) tree[n + i] = a[i];
        for (int i = n - 1; i > 0; --i) tree[i] = merge(tree[i << 1], tree[(i << 1) | 1]);
    }

    void update(int i, int x) {
        tree[i += n] = x;
        while (i >>= 1) tree[i] = merge(tree[i << 1], tree[(i << 1) | 1]);
    }

    int query(int i, int j) {
        int l = 0, r = 0;
        for (i += n, j += n + 1; i < j; i >>= 1, j >>= 1) {
            if (i & 1) l = merge(l, tree[i++]);
            if (j & 1) r = merge(tree[--j], r);
        }
        return merge(l, r);
    }
};
```

sparse-segment-tree.cpp

```cpp
struct sparse_segment_tree {
private:
    struct node {
        int x = 0;
        int l = 0, r = 0;
    };

    array<int, 3> root;
    vector<node> tree;

    int modify(int i, int x, int p, int l, int r) {
        if (i < l or r < i) return p;

        if (not p) {
            p = tree.size();
            tree.emplace_back();
        }

        if (l == r) {
            tree[p].x += x;
            return p;
        }

        int m = l + (r - l) / 2;
        tree[p].l = modify(i, x, tree[p].l, l, m);
        tree[p].r = modify(i, x, tree[p].r, m + 1, r);
        tree[p].x = tree[tree[p].l].x + tree[tree[p].r].x;
        return p;
    }

    int query(int ql, int qr, int p, int l, int r) {
        if (not p or (qr < l or r < ql)) return 0;

        if (ql <= l and r <= qr) return tree[p].x;

        int m = l + (r - l) / 2;
        return query(ql, qr, tree[p].l, l, m) + query(ql, qr, tree[p].r, m + 1, r);
    }

public:
    sparse_segment_tree(int l, int r) {
        root = {0, l, r};
        tree.reserve(2e5);
        tree.emplace_back();
    }

    void modify(int i, int x) {
        root[0] = modify(i, x, root[0], root[1], root[2]);
    }

    int query(int ql, int qr) {
        return query(ql, qr, root[0], root[1], root[2]);
    }
};
```